

# ALLEVAMENTO MANAGER

Documentazione Tecnica Completa

---

Versione 1.0 · Aprile 2026  
React 18 · Supabase · TailwindCSS

# 1. Panoramica del Progetto

Allevamento Manager è una **Progressive Web App (PWA)** pensata per allevatori professionali. Consente di gestire in modo centralizzato cani, cuccioli, accoppiamenti, finanze, salute, calendario eventi ed esposizioni, con dati sincronizzati in cloud tramite Supabase.

|               |                                                   |
|---------------|---------------------------------------------------|
| Tipologia     | Progressive Web App (PWA)                         |
| Framework     | React 18.2 + Vite 7.3                             |
| Backend       | Supabase (PostgreSQL + Auth + Storage)            |
| Styling       | TailwindCSS 3.4                                   |
| Data layer    | TanStack React Query 5 + Zustand 4                |
| Deploy target | Browser moderni (ES2020+), installabile su mobile |

# 2. Stack Tecnologico

## 2.1 Dipendenze Principali

| Pacchetto             | Versione  | Ruolo                  |
|-----------------------|-----------|------------------------|
| react + react-dom     | 18.2.0    | Framework UI           |
| react-router-dom      | 6.21.1    | Routing SPA            |
| @tanstack/react-query | 5.17.9    | Server state & caching |
| zustand               | 4.4.7     | Client state globale   |
| @supabase/supabase-js | 2.39.0    | Backend BaaS           |
| react-hook-form       | 7.49.3    | Gestione form          |
| zod                   | 3.22.4    | Validazione schema     |
| tailwindcss           | 3.4.1     | Utility CSS            |
| lucide-react          | 0.303.0   | Icon library           |
| react-hot-toast       | 2.4.1     | Notifiche toast        |
| date-fns              | 3.0.6     | Manipolazione date     |
| recharts              | 2.15.4    | Grafici                |
| jsPDF + html2canvas   | 4.2 / 1.4 | Export PDF             |
| vite                  | 7.3.1     | Build tool             |

| Pacchetto       | Versione | Ruolo       |
|-----------------|----------|-------------|
| vite-plugin-pwa | 1.2.0    | PWA support |

## 2.2 Variabili d'Ambiente

```
VITE_SUPABASE_URL=https://[project-id].supabase.co
VITE_SUPABASE_ANON_KEY=ey...
```

## 2.3 Script NPM

| Comando         | Descrizione                             |
|-----------------|-----------------------------------------|
| npm run dev     | Avvia server di sviluppo con hot-reload |
| npm run build   | Build produzione in dist/               |
| npm run preview | Preview build produzione in locale      |
| npm run lint    | Analisi statica con ESLint              |

# 3. Struttura del Progetto

```
src/
  pages/           # Pagine applicazione (una per route)
  components/      # Componenti riutilizzabili
    layout/        # Header, Sidebar, Layout wrapper
    dogs/          # Card, Form, PDF, Misurazioni, Grafico
    breeding/      # LitterCard/Form, MatingCard/Form
    puppies/       # PuppyCard, PuppyForm
    finance/       # TransactionCard, TransactionForm
    health/        # HealthRecordCard, HealthRecordForm
    calendar/      # EventForm, EventCard
  contexts/        # AuthContext
  hooks/           # useNotifications
  lib/
    supabase.js    # Client Supabase + tutte le funzioni DB
  App.jsx          # Root component + definizione rotte
  main.jsx         # Entry point
```

# 4. Routing e Navigazione

Il routing è gestito da **React Router DOM v6**. Tutte le rotte (tranne `/auth`) sono protette da **ProtectedRoute**, che verifica la sessione tramite `AuthContext` e reindirizza a `/auth` se l'utente non è autenticato.

| Percorso   | Componente    | Protezione | Descrizione                            |
|------------|---------------|------------|----------------------------------------|
| /auth      | Auth.jsx      | Pubblica   | Login / Registrazione / Reset password |
| /dashboard | Dashboard.jsx | Privata    | Panoramica allevamento                 |
| /dogs      | Dogs.jsx      | Privata    | Lista e filtro cani                    |
| /dogs/:id  | DogDetail.jsx | Privata    | Dettaglio cane con tab                 |
| /breeding  | Breeding.jsx  | Privata    | Accoppiamenti e cucciolate             |
| /puppies   | Puppies.jsx   | Privata    | Gestione cuccioli                      |
| /finance   | Finance.jsx   | Privata    | Entrate e uscite annuali               |
| /health    | Health.jsx    | Privata    | Registro sanitario                     |
| /calendar  | Calendar.jsx  | Privata    | Calendario eventi                      |
| /judges    | Judges.jsx    | Privata    | Giudici ed esposizioni                 |
| /settings  | Settings.jsx  | Privata    | Preferenze e profilo                   |

## 4.1 Configurazione React Query

Il **QueryClient** è configurato con:

- **staleTime:** 5 minuti — i dati non vengono refetchati per 5 min
- **refetchOnWindowFocus:** false — nessun refetch automatico al focus

# 5. Pagine dell'Applicazione

## 5.1 Dashboard

Homepage post-login con panoramica dell'allevamento.

- 4 stat card: Cani Attivi, Cuccioli Disponibili, Incassi Annuali, Spese Annuali
- Bilancio annuale con indicatore profit/loss
- Sezione «Prossimi Eventi» (5 eventi non completati)
- Sezione «Transazioni Recenti» (mese corrente)
- Modal inline per modifica evento e transazione

## 5.2 Dogs — Lista Cani

Gestione della lista completa dei cani dell'allevamento.

- Ricerca full-text per nome, razza, microchip
- Filtro per stato (attivo / venduto / ceduto)

- Grid di DogCard con azioni edit/delete
- Bottone «Aggiungi Cane» → apre DogForm modale
- Empty state con CTA appropriata

## 5.3 DogDetail — Dettaglio Cane

Pagina principale di gestione di un singolo cane, con 6 tab.

- Tab Informazioni: pedigree, microchip, note
- Tab Crescita: misurazioni peso/altezza + grafico Recharts
- Tab Salute: record sanitari (vaccini, visite, terapie)
- Tab Calori (solo femmine): cicli reali e stimati, statistiche interciclo
- Tab Giudici: giudizi delle esposizioni con qualifica e testo
- Tab Storia: timeline cronologica di eventi e cicli
- Export PDF del profilo tramite DogPdfModal

## 5.4 Breeding — Riproduzione

Gestione di accoppiamenti e cucciolate.

- 2 tab: Cucciolate | Accoppiamenti
- Sub-tab cucciolate: attuali (< 6 mesi) e passate
- Ricerca per nome cane madre/padre
- LitterCard e MatingCard con edit/delete
- Form modale per nuova cucciolata o accoppiamento

## 5.5 Puppies — Cuccioli

Tracciamento e gestione dei cuccioli per cucciolata.

- Raggruppamento per cucciolata con header collassabile
- Filtri stato: Tutti / Disponibili / Prenotati / Venduti / Deceduti
- 4 stat card di riepilogo
- PuppyCard con edit/delete
- Ricerca interna per nome cucciolo

## 5.6 Finance — Finanze

Report e inserimento di entrate e uscite.

- Navigazione anno precedente/successivo/corrente
- 3 stat card: Entrate, Uscite, Bilancio
- Filtro per tipo transazione (Tutti / Entrate / Uscite)
- Lista cronologica con TransactionCard
- Cache query 5 minuti per ridurre richieste DB

## 5.7 Health — Salute

Registro sanitario aggregato su tutti i cani.

- 4 tab: Tutti / Vaccini / Visite / Terapie
- 3 stat card: Vaccini Scaduti, Visite Programmate, Terapie Attive
- Grid HealthRecordCard ordinata per scadenza
- Calcolo automatico scadenza da data + giorni intervallo
- Form modale per nuovo record sanitario

## 5.8 Calendar — Calendario

Calendario mensile per la gestione degli appuntamenti.

- Grid mensile con badge colorati per categoria evento
- Click su giorno → form nuovo evento pre-compilato con la data
- Click su evento → dettaglio con toggle completamento
- Legenda categorie: Veterinario, Toelettatura, Esposizione, Calore, Parto, Altro
- Edit/delete direttamente dal dettaglio evento
- Reminder automatici per eventi imminenti

## 5.9 Judges — Giudici

Gestione anagrafica giudici e giudizi delle esposizioni.

- 2 tab: Giudici | Tutti i Giudizi
- 3 stat card: Giudici Registrati, Giudizi Totali, Eccellenti
- Tab Giudici: ricerca, card con conteggio giudizi + bottone «+ Giudizio»
- Tab Giudizi: filtri per giudice e qualifica + bottone «Aggiungi giudizio»
- Qualifiche: Eccellente, Molto Buono, Buono, Sufficiente, Qualificato, Fuori Concorso, Non Classificato
- JudgeModal (create/update giudice), JudgmentModal (create/update giudizio)

## 5.10 Settings — Impostazioni

Preferenze personali e dati dell'allevamento.

- Sidebar con 4 sezioni: Profilo, Allevamento, Notifiche, Sicurezza
- Profilo: nome, email (readonly), telefono
- Allevamento: nome, affisso, partita IVA, indirizzo, sito web
- Sicurezza: cambio password, eliminazione account con doppia conferma
- Persistenza tramite tabella settings su Supabase

## 6. Componenti Riutilizzabili

### 6.1 Layout

| Componente  | Descrizione                                             |
|-------------|---------------------------------------------------------|
| Layout.jsx  | Wrapper principale con Header, Sidebar e area contenuto |
| Header.jsx  | Navbar superiore: logo, user menu, logout               |
| Sidebar.jsx | Menu laterale collassabile con icone di navigazione     |

### 6.2 Dogs

| Componente          | Descrizione                                                                          |
|---------------------|--------------------------------------------------------------------------------------|
| DogCard.jsx         | Card cane: foto/avatar, info base, barra colore in fondo (flex-col per allineamento) |
| DogForm.jsx         | Form modale create/update cane (tutti i campi + color picker)                        |
| DogProfile.jsx      | Header dettaglio cane con avatar, status badge, sesso                                |
| DogMeasurements.jsx | Tab misurazioni peso/altezza con add/edit/delete                                     |
| DogGrowthChart.jsx  | Grafico crescita con Recharts (peso e altezza)                                       |
| DogPdfModal.jsx     | Export PDF profilo cane via html2canvas + jsPDF                                      |

### 6.3 Breeding, Puppies, Finance, Health, Calendar

| Componente                          | Descrizione                                       |
|-------------------------------------|---------------------------------------------------|
| LitterCard / LitterForm             | Card e form cucciolata                            |
| MatingCard / MatingForm             | Card e form accoppiamento                         |
| PuppyCard / PuppyForm               | Card e form cucciolo                              |
| TransactionCard / TransactionForm   | Card e form transazione finanziaria               |
| HealthRecordCard / HealthRecordForm | Card e form record sanitario                      |
| EventForm / EventCard               | Form e card per eventi calendario                 |
| ProtectedRoute.jsx                  | HOC: reindirige a /auth se utente non autenticato |

## 7. Schema Database (Supabase)

---

Tutte le tabelle hanno **Row Level Security (RLS)** abilitata. Ogni record è isolato per utente tramite la colonna `user_id` riferita a `auth.users`.

### 7.x dogs — Cani

`id, user_id, name, breed, gender, birth_date, status, color, weight, height, microchip, pedigree_number, notes`

### 7.x heat\_cycles — Cicli estri

`id, user_id, dog_id, start_date, end_date, duration_days, notes`

### 7.x matings — Accoppiamenti

`id, user_id, female_id, male_id, mating_date, litter_birth_date, gestation_days, notes`

### 7.x litters — Cucciolate

`id, user_id, mating_id, birth_date, females, males, total_puppies, alive_puppies`

### 7.x puppies — Cuccioli

`id, user_id, litter_id, name, gender, color, status, price`

### 7.x expenses — Spese

`id, user_id, dog_id, category, description, amount, expense_date`

### 7.x income — Entrate

`id, user_id, puppy_id, category, description, amount, income_date`

### 7.x health\_records — Sanità

`id, user_id, dog_id, record_type, description, record_date, next_appointment_date`

### 7.x events — Calendario

`id, user_id, title, description, event_date, event_type, dog_ids[], completed, reminder_days`

### 7.x judges — Giudici

`id, user_id, name, nationality, specialization, notes`

### 7.x dog\_judgments — Giudizi

`id, user_id, dog_id, judge_id, judgment_date, show_name, grade, judgment_text, our_comments`



## **7.x dog\_measurements — Misurazioni**

*id, user\_id, dog\_id, measurement\_date, weight, height*

## **7.x settings — Impostazioni**

*id, user\_id, kennel\_name, kennel\_affix, owner\_name, phone, address, vat\_number, website*

## 8. Funzioni Database (supabase.js)

Il file **src/lib/supabase.js** esporta l'oggetto *db* con tutti i metodi asincroni. Ogni metodo di scrittura chiama internamente *getCurrentUserId()* per recuperare l'UID dall'auth Supabase e aggiunge automaticamente *user\_id* al payload.

| Gruppo        | Funzione                      | Descrizione                           |
|---------------|-------------------------------|---------------------------------------|
| Cani          | getDogs(status)               | Lista cani con filtro stato opzionale |
| Cani          | getDog(id)                    | Cane singolo per ID                   |
| Cani          | createDog(dog)                | Inserisce nuovo cane                  |
| Cani          | updateDog(id, updates)        | Aggiorna campi cane                   |
| Cani          | deleteDog(id)                 | Elimina cane                          |
| Cicli         | getHeatCycles(dogId)          | Cicli estri di un cane                |
| Cicli         | createHeatCycle(cycle)        | Nuovo ciclo estro                     |
| Finanze       | getExpenses(filters)          | Spese con filtri data/categoria       |
| Finanze       | getIncome(filters)            | Entrate con filtri data/categoria     |
| Finanze       | getYearlyIncome(year)         | Totale entrate anno                   |
| Finanze       | getYearlyExpenses(year)       | Totale spese anno                     |
| Cuccioli      | getPuppies(litterId,status)   | Lista cuccioli con filtri             |
| Cucciolate    | getLitters()                  | Cucciolate con join mating            |
| Accoppiamenti | getMatings()                  | Lista accoppiamenti                   |
| Misurazioni   | getDogMeasurements(dogId)     | Misurazioni ordinate per data         |
| Salute        | getHealthRecords(dogId)       | Record sanitari per cane              |
| Eventi        | getEvents(filters)            | Eventi calendario con filtri          |
| Giudici       | getJudges()                   | Lista giudici ordinata per nome       |
| Giudizi       | getAllJudgments()             | Tutti i giudizi con join judge+dog    |
| Giudizi       | getJudgments(dogId)           | Giudizi di un cane specifico          |
| Storage       | uploadImage(file,bucket,path) | Upload foto su Supabase Storage       |
| Stats         | getActiveDogs()               | Count cani attivi                     |
| Stats         | getAvailablePuppies()         | Count cuccioli disponibili            |
| Settings      | getSettings()                 | Preferenze utente corrente            |
| Settings      | updateSettings(updates)       | Salva preferenze                      |

## 9. Autenticazione

---

L'autenticazione è gestita da **Supabase Auth** con sessione persistita nel browser. Il **AuthContext** espone il hook `useAuth()` disponibile in tutta l'app.

### 9.1 Metodi supportati

- **Email + Password** — login e registrazione standard
- **OAuth Google** — login con account Google
- **Password Reset** — invio link di reset via email

### 9.2 useAuth() Hook

|                  |                                                                    |
|------------------|--------------------------------------------------------------------|
| <b>user</b>      | Oggetto utente Supabase (null se non autenticato)                  |
| <b>loading</b>   | true durante il controllo iniziale della sessione                  |
| <b>signOut()</b> | Effettua il logout e reindirizza a /auth                           |
| <b>isPrivate</b> | true se <code>user.user_metadata.account_type === 'privato'</code> |

### 9.3 Registrazione

Al momento della registrazione vengono eseguiti automaticamente:

- Creazione utente in Supabase Auth
- Inserimento record nella tabella `settings` con `kenel_name`

## 10. Funzionalità Avanzate

---

### 10.1 Export PDF Profilo Cane

Implementato in **DogPdfModal.jsx** usando la combinazione `html2canvas` + `jsPDF`:

- Rendering DOM del profilo cane in un div nascosto
- `html2canvas` converte il DOM in canvas PNG
- `jsPDF` inserisce il canvas come immagine nel PDF
- `jspdf-autotable` per eventuali tabelle strutturate

### 10.2 Tracciamento Cicli Estri

La tab **Calori** (solo femmine) offre:

- Registro cicli reali con data inizio/fine e durata
- Stima automatica del prossimo calore basata sulla media degli intercicli
- Sincronizzazione automatica con il Calendario: crea/aggiorna evento di tipo *calore\_stimato*
- Calcolo automatico parto stimato (63 gg) al salvataggio di un accoppiamento

## 10.3 Grafici di Crescita

Implementati con **Recharts** in *DogGrowthChart.jsx*: LineChart dual-axis per peso (kg) e altezza (cm) nel tempo, con tooltip e legenda.

## 10.4 PWA

Configurata tramite **vite-plugin-pwa**: l'app è installabile su dispositivi mobili e desktop, con service worker per caching delle risorse statiche.

## 10.5 Notifiche Push

Integrazione **OneSignal** tramite il hook *useNotifications.js* per notifiche push su eventi imminenti (scadenze vaccini, calori stimati, appuntamenti).

# 11. Sicurezza

---

## 11.1 Row Level Security (RLS)

Tutte le tabelle Supabase hanno RLS abilitata con policy che isolano i dati per utente:

```
-- Esempio policy SELECT
create policy "Utente vede solo i propri dati"
  on judges for select using (auth.uid() = user_id);

-- Esempio policy INSERT
create policy "Utente inserisce propri dati"
  on judges for insert with check (auth.uid() = user_id);
```

## 11.2 Protezione Rotte

Il componente **ProtectedRoute** verifica la sessione ad ogni navigazione. Se la sessione è scaduta o assente, l'utente viene reindirizzato automaticamente a */auth*.

## 11.3 Storage

Le immagini vengono caricate su Supabase Storage in bucket isolati per utente (path: *userId/path/filename*). Le chiavi di accesso sono gestite lato Supabase e mai esposte nel codice client.

## 12. Build e Deploy

---

|                         |                                                 |
|-------------------------|-------------------------------------------------|
| <b>Build command</b>    | npm run build                                   |
| <b>Output directory</b> | dist/                                           |
| <b>Node minimo</b>      | Node.js 18+                                     |
| <b>Browser target</b>   | ES2020+ (Chrome, Firefox, Safari, Edge moderni) |
| <b>Env vars</b>         | VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY       |

Il progetto è compatibile con qualsiasi hosting statico (Vercel, Netlify, Cloudflare Pages). Il file *vite.config.js* include il plugin PWA per la generazione automatica del Service Worker e del manifest.

# Appendice — SQL Setup Tabelle Principali

---

Eseguire nel **SQL Editor** di Supabase per creare le tabelle giudici e giudizi:

```
-- Tabella giudici
create table if not exists judges (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid not null references auth.users(id) on delete cascade,
  name        text not null,
  nationality  text,
  specialization text,
  notes       text,
  created_at  timestampz default now()
);
alter table judges enable row level security;
create policy "sel" on judges for select using (auth.uid() = user_id);
create policy "ins" on judges for insert with check (auth.uid() = user_id);
create policy "upd" on judges for update using (auth.uid() = user_id);
create policy "del" on judges for delete using (auth.uid() = user_id);

-- Tabella giudizi
create table if not exists dog_judgments (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid not null references auth.users(id) on delete cascade,
  dog_id       uuid not null references dogs(id) on delete cascade,
  judge_id     uuid not null references judges(id) on delete cascade,
  judgment_date date not null,
  show_name    text,
  grade        text,
  judgment_text text,
  our_comments text,
  created_at   timestampz default now()
);
alter table dog_judgments enable row level security;
create policy "sel" on dog_judgments for select using (auth.uid() = user_id);
create policy "ins" on dog_judgments for insert with check (auth.uid() = user_id);
create policy "upd" on dog_judgments for update using (auth.uid() = user_id);
create policy "del" on dog_judgments for delete using (auth.uid() = user_id);
```